THE LAYER IS NOT ATOMIC: PARTITIONING THE RESIDUAL STREAM FOR COMPUTE-EFFICIENT TRANSFORMERS

Anonymous authors
Paper under double-blind review

ABSTRACT

The transformer layer is monolithic by convention: a single attention block and a single feed-forward network, both of width D, process every token at every layer. A growing line of work decomposes this layer into several narrower parallel blocks, but every such design pays for the narrowing with learned down- and up-projections into and out of the reduced width, and buys back the cost with routing sparsity. We show the payment is avoidable. In the *Modular Sub-Transformer* (MST), each layer contains N parallel sub-transformers of width $d = D/N$ that *partition* the residual stream: the N sub-states, concatenated, *are* the D-dimensional representation, so no projection is required to enter or leave a narrow block. Sub-transformers exchange information once per layer through a routed d-dimensional bottleneck with a relu^2 expansion. Because the per-sub head budget satisfies $n_h \cdot d_h = d$, attention FLOPs and KV cache size are preserved *exactly*, while per-layer matmul parameters fall by $3.20\times$ at $N{=}4$. The model remains fully dense: every sub-transformer runs for every token, so there is no expert dispatch, no capacity constraint and no load-balancing sparsity to tune. Against a strongly-tuned dense baseline (Muon, μP, QK-norm, value embeddings, sliding-window attention) on a compute-optimal token ladder, MST is more compute-efficient over $413\mathrm{M} \to 964\mathrm{M}$ total parameters: matching MST's bits-per-byte requires $1.23\times$ the inference FLOPs per token or $1.10\times$ the training compute. Matrix parameters are unchanged within measurement ($0.92\times$); the saving is in compute, not parameter count. **[TODO: add one downstream + one wall-clock number to the abstract once Tables 4 and 5 are filled]**

1 INTRODUCTION

Scaling a transformer language model means growing two numbers: depth L and width D. The layer itself is treated as an indivisible unit: one attention block and one FFN, both spanning the full D channels. Every token is processed by all D^2-scale weight matrices at every layer, and no structural distinction is drawn between different parts of the representation.

Recent work has begun to question this. Mixture-of-Experts (Shazeer et al., 2017; Fedus et al., 2022) routes tokens to alternative FFNs, but each expert still reads the full D-dimensional token and the layer remains full-width. Layer-level modularity, as in Mixture-of-Depths (Raposo et al., 2024), MoEUT (Csordás et al., 2024) and Mixture-of-Recursions (Bae et al., 2025), changes *which* full-width blocks execute. Most directly, Mixture-of-Layers (MoL) (Ternovtsii & Bilak, 2026b) replaces full-width blocks with K parallel *thin* blocks of width $d_{\text{thin}} \ll D$, composed by top-k routing.

MoL isolates the right question and, in our reading, also exposes the cost of the standard answer. Because its thin blocks must read from and write to a full-width residual stream, each is wrapped in learned projections $W_{\text{down}} \in \mathbb{R}^{d_{\text{thin}} \times D}$ and $W_{\text{up}} \in \mathbb{R}^{D \times d_{\text{thin}}}$. That wrapper consumes 40% of block parameters at $d_{\text{thin}}{=}256$ and 73% at $d_{\text{thin}}{=}64$. This is a *projection tax* that grows precisely as the blocks get narrow enough to be interesting. The tax is paid back with sparsity: only k of K blocks run per token, which recovers active-compute efficiency but forfeits total-parameter efficiency. At 1.3B scale MoL reports a 3.01 PPL deficit against an iso-total-parameter dense baseline, and a 1.9–$2.3\times$ training-time premium.

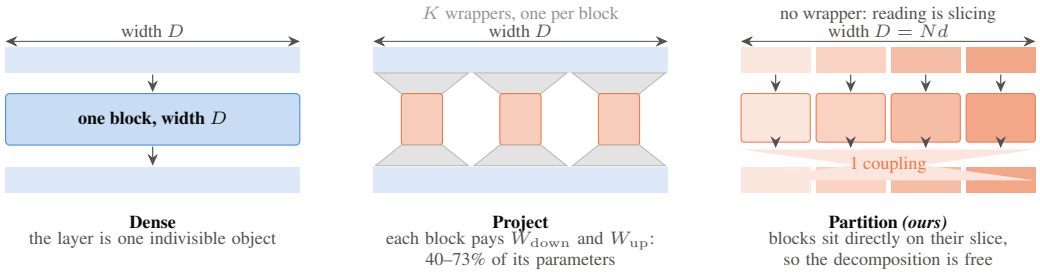

Figure 1: Three ways to make a transformer block narrow, all drawn at the same total width D. Projecting into a narrow block costs a $W_{\text{down}}/W_{\text{up}}$ wrapper on *every* block. Partitioning costs nothing, because each block sits directly on its own slice of the stream and the N slices concatenated are the representation. The single coupling that replaces the wrappers is detailed in Figure 2.

We ask whether the projection can be removed rather than amortized. Our answer is a change of viewpoint: instead of projecting a full-width residual stream *into* narrow blocks, let the narrow blocks *be* the residual stream. If a layer holds N sub-transformers of width $d = D/N$, and sub-transformer i owns channels $[id, (i{+}1)d)$ of the representation, then reading and writing are free: they are slicing and concatenation. The decomposition costs nothing, and what remains to be designed is only how the sub-transformers communicate.

Our contributions:

1. **A projection-free layer decomposition.** MST partitions the residual stream into N sub-streams processed by independent width-d transformers, coupled once per layer by a routed bottleneck (Section 3). Per-layer matmul parameters fall from $12D^2$ to $3.75D^2$ at $N{=}4$, while attention FLOPs and KV cache are preserved *exactly by construction*, not approximately.
2. **Compute efficiency at parameter parity.** On a compute-optimal ladder from 413M to 964M total parameters, MST is more efficient than a strong dense baseline on both compute axes, inference FLOPs per token ($1.23\times$) and total training compute ($1.10\times$), while matrix parameters are at parity ($0.92\times$) (Section 5). The saving is in compute per parameter, not in parameter count, and we are explicit that the parameter axes are not a win for this configuration.
3. **Separation from the closest concurrent architecture, on four independent grounds.** MoL (Ternovtsii & Bilak, 2026b) reaches the same question by wrapping each narrow block in its own projection pair rather than partitioning. We reimplement it from its published equations and show that the difference is structural, not incidental: its total-to-active matrix ratio is 3.74 against MST's 1.48 and is flat across depth; at equal training data MST is above the dense curve on FLOPs per token where MoL is below it, at $0.67\times$ its parameters; at matched quality MST's prefill overtakes dense beyond $T{\approx}190$K, reaching $1.31\times$ at $T{=}524288$ on $4.3\times$ less peak memory, because its windowed streams scale as $T^{1.59}$ against dense's $T^{1.85}$, where MoL reports its own sparse path losing to dense; and MoL's central mechanism, gating attention, *costs* $1.86\times$ what it saves when transplanted into MST (Section 5.4).
4. **The optimization of partitioned weights matters as much as the architecture.** A naively-trained partition suffers a $7.2\times$ imbalance in per-sub gradient norm and cross-sub contamination in the orthogonalizing optimizer. Two fixes, per-sub gradient-norm equalization and block-diagonal Newton–Schulz, together with a widened transition bottleneck, recover -0.11 BPB at $D{=}512$ (Section 6).
5. **Coupling is necessary; its topology is not.** Removing the learned transition costs 0.039 BPB, yet eleven distinct attempts to enrich it (nonlinear, gated, MLP, bilinear, per-slice routed, multi-layer lookback, cross-sub gating, cross-sub KV injection, low-rank query modulation, feature cycling, and a global residual stream) all fail to improve on it (Section 6). This is an independent dense-architecture analogue of the routing-equifinality result of Ternovtsii & Bilak (2026a).

2 Related Work

Narrow parallel blocks. MoL (Ternovtsii & Bilak, 2026b) is the closest work and the sharpest contrast. It replaces full-width blocks with K thin transformer blocks at reduced d_{thin}, selected per token by top-k routing, and adds a shared full-attention block because sparse dispatch leaves each routed block seeing only a fraction of the sequence. The design is motivated by the same observation as ours, that the layer need not be monolithic, but it answers differently on three counts. Its thin blocks read from and write to a full-width residual stream, so each carries a $W_{\text{down}}/W_{\text{up}}$ wrapper costing 40–73% of the block; it recovers that cost through sparsity, activating k of K blocks; and it substitutes linear attention in the routed blocks. The consequences are visible in its own results: at 1.3B, MoL leads an iso-active dense baseline by 0.49 PPL but trails an iso-*total* one by 3.01 PPL, and trains 1.9–2.3$\times$ slower. MST removes the wrapper instead of amortizing it, stays dense, and keeps softmax attention throughout, which is why it can be compared on total parameters at all. UoE (Yang et al., 2025) decomposes attention and MLP into expert groups with hierarchical routing, and shares MoL's reliance on a full-width stream and on sparsity.

Residual-stream structure. Hyper-Connections (Zhu et al., 2025; Xie et al., 2026) expand the residual into n *full-width* streams to improve optimization, at roughly $n\times$ residual memory; AltUp (Baykal et al., 2023) widens the token representation and updates one block per layer; Multi-Stream Transformers (Burtsev & Rumshisky, 2021) split an encoder into parallel streams merged at the end. All of these *widen*. MST *narrows* N streams to D/N and spends the saving on depth. Concurrent work on FFN shape (Anonymous, 2026b) and channelized architectures (Anonymous, 2026a) restructures within a layer but leaves the residual width convention intact.

Partial partitions. MH-MoE (Wu et al., 2024) splits tokens into sub-tokens routed to different experts and FlashMHF (Zhang et al., 2026) builds parallel narrow sub-networks inside the FFN; both are FFN-only and re-merge within the layer. Layer-level modularity (Raposo et al., 2024; Csordás et al., 2024; Bae et al., 2025) changes which full-width blocks execute rather than their width. MST partitions attention as well, and the partition persists across the full depth.

Structured matrices and attention spans. An MST layer is by construction a block-diagonal weight matrix with N blocks plus a low-rank cross-block correction, placing it in the family of Monarch (Dao et al., 2022) and BLAST (Lee et al., 2024), but applied to a whole layer rather than to individual linear maps and with a learned, input-dependent correction. Assigning different context windows to different heads or layers is likewise established (Sukhbaatar et al., 2019; Beltagy et al., 2020; Fu et al., 2024; Zhang et al., 2025); MST lifts it from the head to the stream, so a window governs a block's attention, its FFN and its slice of the residual jointly.

3 Method

Notation. D is the model width, N the number of sub-transformers, $d = D/N$ the sub width, and L the depth. The state at layer ℓ is $H^{(\ell)} \in \mathbb{R}^{B\times T\times N\times d}$; its flattening over the last two axes is the ordinary D-dimensional residual stream. We write $X_i \in \mathbb{R}^d$ for the i-th sub-state at a given token and suppress the batch and time axes throughout. Each sub-transformer uses n_h heads of dimension d_h with $n_h d_h = d$.

3.1 The layer

Per stream: nothing new. Sub-transformer i owns X_i and no other channel. It runs an ordinary pre-norm transformer block at width d, with its own attention window w_i, RoPE (Su et al., 2021) and QK-norm:

$$X_i \leftarrow X_i + W_i^O \,\mathrm{Attn}_{w_i}\big(W_i^Q \tilde{X}_i,\, W_i^K \tilde{X}_i,\, W_i^V \tilde{X}_i\big), \qquad X_i \leftarrow X_i + W_i^{F2}\,\mathrm{relu}^2\big(W_i^{F1}\tilde{X}_i\big), \quad (1)$$

where $\tilde{X}_i = \mathrm{RMSNorm}(X_i)$ is recomputed before each sublayer. This is deliberately unremarkable: it is the dense baseline's block with D replaced by d everywhere. Because no term reads a channel that sub i does not own, the N blocks together form a block-diagonal factorization of one D-width layer rather than N copies of a smaller model.

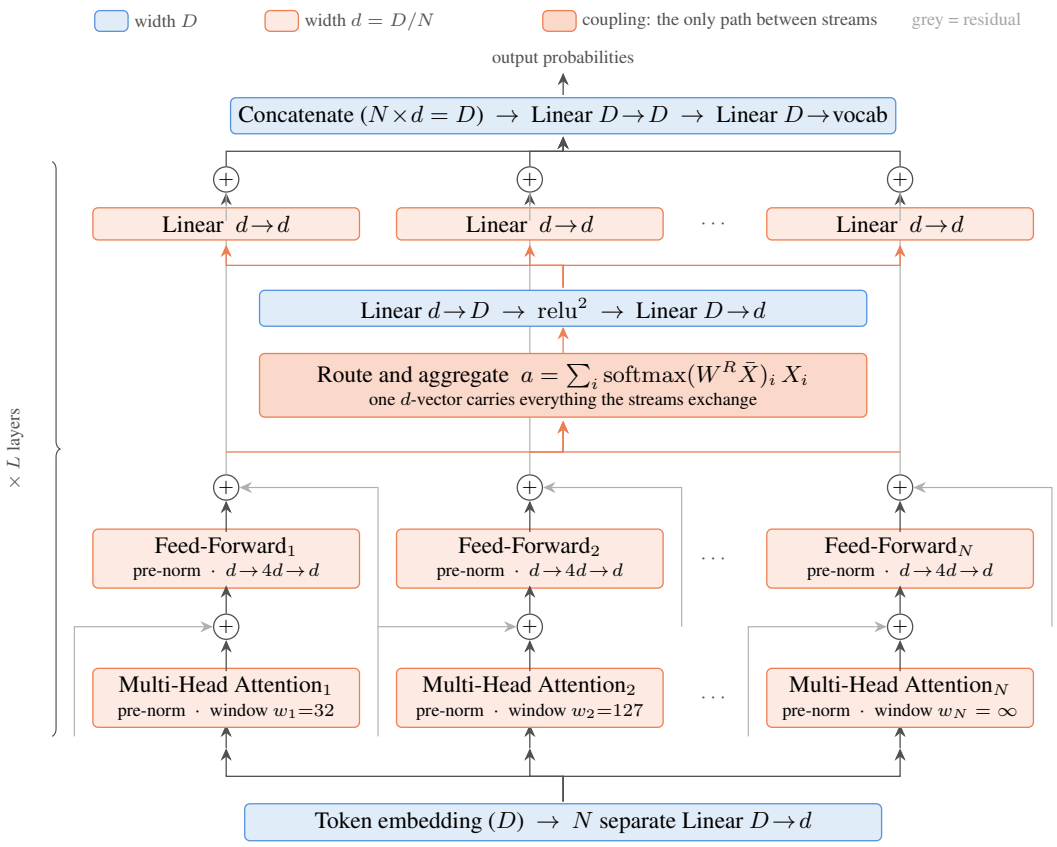

Figure 2: One MST layer, read bottom to top. Colour encodes width and nothing else: blue operates at D, orange at $d = D/N$. The N streams run ordinary transformer sublayers in parallel and never read each other's channels; the coupling is the only path between them, and everything they exchange passes through the single d-vector a. Grey lines are residual connections, which carry each stream past the coupling unchanged.

Coupling: the whole mechanism, in one line. The streams exchange information exactly once per layer. Everything they say to each other passes through a single d-dimensional vector:

$$X_i \leftarrow X_i + \overbrace{W_i^D}^{\text{scatter } 1 \to N} \underbrace{W^{\downarrow}\,\text{relu}^2\Big(W^{\uparrow}}_{\text{bottleneck } d \to D \to d} \overbrace{\textstyle\sum_j p_j \tilde{X}_j}^{\text{aggregate } N \to 1}\Big), \qquad p = \text{softmax}\Big(W^R \tfrac{1}{N} \textstyle\sum_j \tilde{X}_j\Big). \tag{2}$$

Read right to left, Eq. 2 is the hourglass of Figure 2: N streams collapse to one d-vector, that vector is widened to D for a single nonlinearity and squeezed back to d, and N distinct read-out matrices deliver it. Parameter shapes and counts are in Appendix A.

Three consequences follow directly from the form of Eq. 2.

(i) The router selects nothing. p lies in the simplex over *sub-transformers*, not over tokens-to-experts. Every stream runs for every token, so MST has no sparse dispatch, no capacity factor, and no dropped tokens. A light load-balancing term $\alpha N \sum_i \bar{p}_i^2$ with α=0.01 (Fedus et al., 2022) is minimized at uniform p.

(ii) The N read-outs are what break the symmetry. Every stream receives the same vector. Through a shared W^D they would receive identical messages and could diverge only through initialization noise. The distinct W_i^D make the message stream-specific, and they are the cheapest term in the layer (Appendix A).

(iii) The layer is exactly the identity at step 0. W_i^O, W_i^{F2} and $W^{\downarrow}$ are zero-initialized and W^R starts at zero, so every residual branch contributes nothing and p is exactly uniform. Training begins from a well-conditioned deep identity, not from a random partitioned map.

The head budget is conserved. Each sub spends $n_h d_h = d$ on attention, so the layer spends $Nd = D$, exactly as a dense layer does. This one identity is why attention FLOPs and KV cache are preserved while only the matmul parameters shrink (Section 3.4). Our dense baseline uses d_h=128; MST uses d_h=128/N=32 with N times as many heads.

3.2 Multi-scale sub-windows

Sub-transformer i attends with a causal sliding window w_i spaced geometrically from $w_{\min}$=32 to full context, the last sub always unrestricted. For N=4 at T=2048 this gives $w = (32, 127, 511, \infty)$. This is the only component of MST that supplies an *a priori* reason for sub-transformers to differ; every other source of differentiation is symmetry breaking through W_i^D and initialization.

The assignment is close to FLOP-neutral, so it is a prior rather than a shortcut. Summed effective context is $32 + 127 + 511 + 2048 = 2718$ per layer against $4 \times 704 = 2816$ for the dense baseline's `SSSL` layer-alternating pattern, a 3.5% reduction.

3.3 Optimizing partitioned weights

The N matrices of each type are stored fused as one $(N{\cdot}\text{out}) \times$ in tensor and applied with a batched matrix multiply. That is an implementation detail for throughput, but it exposes two training problems that a dense model does not have.

Gradient imbalance. Measured on the N=4 model, per-sub gradient norms differ by 7.2×: sub 0 receives 55% of the total and sub 3 receives 7.6%. The mechanism is a feedback loop through the concatenating output head, which weights one sub more, which trains it faster, which weights it more still. We equalize with a backward hook that views the fused gradient as $(N, \text{out}, \text{in})$ and rescales each sub to the mean norm. Only magnitudes are touched; directions are left alone.

Cross-sub contamination in Muon. Muon orthogonalizes the update via a Newton–Schulz / Polar-Express iteration (Jordan et al., 2024; Amsel et al., 2025). Applied to the fused tensor it mixes gradient directions belonging to different sub-transformers, which is exactly what the partition is designed to keep apart. We run the iteration independently on each of the N blocks. Because Muon scales its step by $\max(1, \text{rows}/\text{cols})^{1/2}$, blocking also divides the effective learning rate on the fused attention tensors by $\sqrt{N}$; we restore it with a per-sub multiplier of $\sqrt{N}$, which simultaneously corrects the μP (Yang et al., 2021) mismatch between a D-tuned rate and d-wide matrices. The two interventions are therefore coupled and we ablate them jointly (Section 6).

3.4 Cost accounting

Parameters. Summing the components of Eq. 2 and the per-stream sublayers (Appendix A gives the term-by-term budget),

$$\underbrace{\tfrac{12D^2}{N}}_{N \text{ sub-transformers}} + \underbrace{\tfrac{3D^2}{N}}_{\text{coupling}} = \frac{15D^2}{N} \quad \text{per layer, against} \quad 12D^2 \text{ dense}, \tag{3}$$

a reduction of $0.8N$ rather than N: the coupling consumes a quarter of what the partition saves. It is nonetheless far cheaper than a projection wrapper, which costs $2Dd$ *per block* rather than once per layer.

Attention FLOPs and KV cache. A sub contributes $12\, n_h d_h\, T_{\text{eff}} = 12\, d\, T_{\text{eff}}$ per token per layer; summed over N subs this is $12\, D\, T_{\text{eff}}$, the dense value exactly. The KV cache is $Nd = D$ per layer, again exactly dense. MST therefore takes its saving entirely from matmul parameters and inherits neither an attention advantage nor an attention penalty, which is what makes the FLOPs-per-token axis of Section 5 a clean comparison.

Table 1: Cost and quality. MST is $N{=}4$ with all components of Section 3. Dense is the matched baseline at the same $D = 64L$ ladder. "Matrices" excludes embeddings and value embeddings. Active counts discount the streams the top-k gate switches off. The gate reaches the FFN only, which is 44.4% of a layer's matmul parameters, so $k{=}1$ of $N{=}4$ removes 33.3% of a layer and 66.7% stays active; gating attention as well would remove 62.5% (Section 8). For dense the active and total counts coincide by definition. The FLOPs/token column is the *active* count for both arms.

		Parameters		Matrices		FLOPs		
Model	L	Total	Active	Total	Active	/token	Training	BPB↓
MST	8	110.6M	107.5M	10.0M	6.8M	1.562e8	4.386e16	1.0320‡
	16	413.2M	388.1M	77.7M	52.5M	5.746e8	6.710e17	0.8702
	20	654.2M	605.0M	150.9M	101.7M	9.528e8	1.929e18	0.8338
	24	964.4M	879.4M	259.7M	174.8M	1.481e9	4.823e18	0.7869
Dense	8	125.8M	125.8M	25.2M	25.2M	2.863e8	1.261e17	0.9602
	12	286.3M	286.3M	84.9M	84.9M	7.597e8	8.537e17	0.8654
	14	399.1M	399.1M	134.9M	134.9M	1.129e9	1.899e18	0.8312
	16	536.9M	536.9M	201.3M	201.3M	1.585e9	3.817e18	0.7988
	18	701.9M	701.9M	286.7M	286.7M	2.180e9	7.268e18	0.7755
	20	896.5M	896.5M	393.2M	393.2M	2.886e9	1.292e19	0.7530
	22	1123.2M	1123.2M	523.4M	523.4M	3.763e9	2.209e19	0.7338
	24	1384.1M	1384.1M	679.5M	679.5M	4.775e9	3.594e19	0.7169
	26	1681.8M	1681.8M	863.9M	863.9M	5.991e9	5.684e19	0.7006
	28	2018.5M	2018.5M	1079.0M	1079.0M	7.366e9	8.661e19	0.6870
	30	2396.7M	2396.7M	1327.1M	1327.1M	8.977e9	1.291e20	0.6744

‡ below the amortization threshold (it sits *above* the dense curve on both compute axes); excluded from the fits of Table 3 and reported as a sensitivity in Section 8. The MST ladder currently has three points above the threshold, against eleven for dense; Section 8 treats this as the principal limitation of the scaling claim. All cost columns are recomputed from a single implementation at one commit, so the two arms are directly comparable; the token budget is 10.5 tokens per scaling parameter and Train FLOPs is FLOPs/token $\times$ tokens.

Value embeddings. Per-layer value embeddings (Zhou et al., 2024) in MST live at width d and are shared across subs with per-sub gates, so they are $N\times$ smaller than the dense baseline's. Since the dense baseline spends 44% of its total parameters on value embeddings at $L{=}24$, we report transformer-matrix parameters alongside total parameters throughout (Table 1) so that this effect can be separated from the layer decomposition itself.

4 Experimental setup

Baseline. Our dense baseline is not a vanilla transformer. It is a speedrun-tuned decoder (Karpathy, 2025) with RMSNorm pre-norm, RoPE, QK-norm, relu^2 FFN, per-layer value embeddings, layer-alternating sliding-window attention, logit soft-capping, learned per-layer residual scalars, Muon for matrices with AdamW for embeddings and scalars, μP learning-rate transfer, and a batch-size schedule following Bergsma et al. (2025). MST changes only the layer structure; every other component, hyper-parameter and schedule is shared.

Ladder. $D = 64L$ rounded up to a multiple of 128; $N{=}4$; $T{=}2048$; vocabulary 32,768. Both arms are trained at 10.5 tokens per scaling parameter (transformer matrices plus output head), so each model receives its own compute-optimal budget (Hoffmann et al., 2022); MST consequently trains on fewer tokens than dense at equal depth. Data is a filtered web corpus (Penedo et al., 2024), single epoch.

Metric. Bits per byte on held-out data, which is invariant to tokenizer vocabulary.

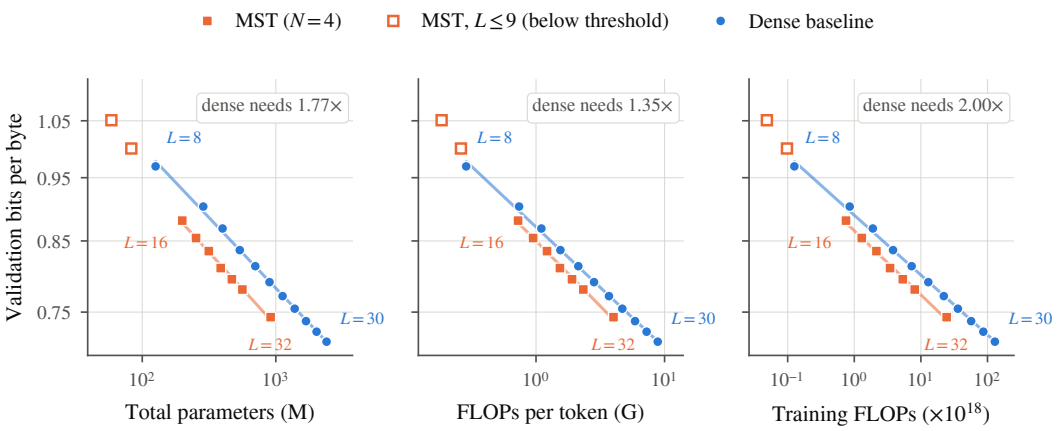

Figure 3: MST lies below the dense scaling curve on both compute axes and above it on the parameter axes. Points are measured runs; lines are power-law fits (Table 3). Hollow markers are MST models below the amortization threshold, which are excluded from the MST fit. Each panel states the multiplier a dense model needs to reach the same bits-per-byte, averaged over $L \in [16, 32]$. MoL (Ternovtsii & Bilak, 2026b) appears on the training-FLOPs and matrix-parameter panels only: under compute-optimal budgeting it trains on $3.13\times$ MST's tokens at L=16, so a per-token point would not be comparable. Section 5.3 gives that comparison at fixed data.

5 Results

Which parameter axis, and what it now says. We report matrix parameters alongside total parameters. Embeddings, per-layer value embeddings and the output head are excluded from the former, following Kaplan et al. (2020) and Hoffmann et al. (2022). The reason is not convention alone: the dense baseline spends 44% of its total parameters on value embeddings at L=24, and the width of that table is a design choice orthogonal to the layer decomposition — widening MST's from d to Nd moves total parameters by $1.95\times$ while leaving matrix parameters bit-identical (Table 12). A parameter curve in which one lookup table moves the answer more than the architecture does is measuring the wrong quantity. We report total parameters throughout so that this can be checked rather than taken on trust.

On the current configuration neither parameter axis is a win. Matrix parameters sit at $0.92\times$ averaged over $L \in [16, 24]$ and $1.01\times$ at L=16, so MST is parameter-neutral on the axis it is responsible for, and total parameters are $0.64\times$, driven by the per-stream value-embedding table (Table 12: the shared-table variant is $1.23\times$ on total parameters at a cost of $0.08\times$ on FLOPs per token). The efficiency claim of this paper is therefore on the two compute axes, not on parameters, and we state it that way throughout. An earlier configuration of MST inverted this trade, winning on parameters and losing on compute; we report the compute-efficient one because compute is the resource the paper is about.

5.1 Pareto dominance

Figure 3 shows MST below the dense curve on both compute axes over the measured range, and above it on both parameter axes. Table 2 quantifies the gap by inverting the dense fit: for each MST model we report the resource a dense model would require to reach the same BPB.

5.2 Scaling exponents

Fitting BPB $= ax^b$ (Table 3), MST's exponents are marginally steeper on all three axes, but the difference is within fit uncertainty for seven points and we do *not* claim a scaling-exponent advantage. The defensible statement is that the multiplier of Table 2 is approximately constant over a $4.6\times$ span of parameters and a $32\times$ span of training compute. Including models below 100M ($L \leq 9$) would

Table 2: Resource multiplier a dense model needs to match each MST model, obtained by inverting the dense power-law fit. The multiplier is stable across the range. Matrix parameters (embeddings, value embeddings and the output head excluded) is the primary parameter axis, following Kaplan et al. (2020); total parameters are reported below the rule for completeness. †FLOPs per token is an inference-cost axis and is only comparable across architectures at fixed training data; see Section 5.3.

MST L	16	20	24	**mean**
FLOPs / token†	1.25	1.15	1.30	**1.23**×
Training FLOPs	1.17	0.94	1.18	**1.10**×
Matrix parameters	1.01	0.83	0.92	**0.92**×
Total parameters	0.66	0.60	0.66	**0.64**×

Table 3: Power-law fits BPB $= ax^b$, MST $L \in [16, 24]$ (three points) and dense $L \in [8, 30]$ (eleven points). The MST R^2 values are materially lower than dense's; with three points these fits are descriptive, not predictive.

Axis x	Dense a, b	R^2	MST a, b	R^2
FLOPs / token†	$7.035\,x^{-0.1025}$	0.999	$7.366\,x^{-0.1057}$	0.984
Training FLOPs	$7.036\,x^{-0.0507}$	0.999	$7.015\,x^{-0.0508}$	0.984
Matrix parameters	$4.444\,x^{-0.0897}$	1.000	$3.924\,x^{-0.0827}$	0.979
Total parameters	$8.894\,x^{-0.1196}$	0.999	$9.072\,x^{-0.1180}$	0.982

widen the exponent gap substantially, but those points lie *above* the dense curve on the compute axes and steepen the fit artificially; we report that crossover in Section 8 rather than fitting through it.

5.3 Iso-token control and comparison to MoL

The compute-optimal protocol of Section 4 gives each architecture a token budget proportional to its own scaling parameters, so arms that carry more matrix parameters train on more data. This is the correct protocol for a compute-efficiency claim, but it confounds any *per-token* comparison, and the confound is large for architectures whose total and active parameter counts diverge. We therefore report an iso-token control.

We reimplement MoL (Ternovtsii & Bilak, 2026b) from its published equations; there is no public code release. Our implementation reproduces the paper's reported parameter counts to within 0.5% (85.3M for its Table 1 configuration, 0.61B active for its 1.3B configuration) and all three of its quoted projection-overhead figures (40/57/73% at $d \in \{256, 128, 64\}$), which is the strongest correctness evidence available without a reference implementation. Routed blocks use softmax attention rather than Gated DeltaNet; the authors' own dense-DeltaNet control matches dense softmax to within 0.01 PPL, so the attention type is not the source of MoL's structural effect, but this does mean we do not run their single best configuration.

At L=8 under the compute-optimal protocol the three arms draw very different budgets, because MoL's 1+3of15 topology carries $3.75\times$ more total than active parameters: 0.59B tokens for MoL, 0.44B for dense, 0.28B for MST. Table 4 repeats all three at a common 0.6B, chosen as MoL's own allocation under this rule so that it is not disadvantaged.

Three observations. First, the protocol confound is most of the apparent gap: MoL leads MST by 0.0412 BPB at each arm's own budget and by 0.0100 at equal tokens, a $4.1\times$ reduction. MST gains 0.0341 from the extra data and MoL only 0.0030, since MoL was already at its budget. This also reconciles our numbers with the source paper, which trains every arm of a comparison at a fixed token count and consequently reports no gain of the size the unequal-budget reading suggests.

Second, the ranking inverts. At equal tokens MST is above the dense curve on FLOPs per token ($1.061\times$) and MoL is below it ($0.962\times$), the reverse of the unequal-budget picture. The local ex-

Table 4: Iso-token control at L=8. Multipliers are anchored on the dense arm of the same table and use the dense ladder exponents, so dense is $1.000\times$ by construction. Both MST and MoL use plain shared value embeddings here, so the only difference is the layer structure.

	BPB		At 0.6B tokens		
Model	own budget	0.6B	FLOPs/token	training FLOPs	total params
Dense	0.9602 (0.44B)	0.9427	$1.000\times$	$1.000\times$	125.8M
MoL 1+3of15	0.9908 (0.59B)	0.9878	$0.962\times$	$0.610\times$	89.7M
MST (N=4)	1.0320 (0.28B)	0.9978	$\mathbf{1.061}\times$	$0.609\times$	**60.3M**

Table 5: Iso-token control at L=16, both arms at 3.658B tokens with their own value-embedding variant. Multipliers are read against the dense ladder fits. The budget is MoL's own allocation under our protocol; MST is $3.13\times$ past its own, which inflates its per-token multiplier and is stated below.

Model	BPB	FLOPs/tok	train FLOPs	act. matrices	tot. matrices	tot. params
MST N=4	0.8281	$\mathbf{1.998}\times$	$0.939\times$	$\mathbf{2.633}\times$	$\mathbf{1.780}\times$	$\mathbf{0.982}\times$
MoL 1+3of15	**0.8062**	$1.849\times$	$\mathbf{1.144}\times$	$2.208\times$	$0.590\times$	$0.367\times$

MST reaches within 0.0119 BPB of MoL while spending 29.0% fewer FLOPs per token. The dense exchange rate prices a 29.0% reduction at $+0.0282$ BPB; MST paid $+0.0119$, a margin of 0.0163 in its favour, so over that interval MST converts FLOPs into quality at roughly $2.4\times$ the dense rate relative to MoL. It also carries $4.05\times$ fewer matrix parameters and trains on $0.71\times$ the compute.

ponent from MST to MoL is -0.052, half the dense ladder's -0.104: MoL spends $1.215\times$ MST's FLOPs per token and returns 0.0100 BPB where the dense curve would return 0.0202 for the same spend. MST reaches this with 60.3M total parameters against MoL's 89.7M.

Third, the two are indistinguishable on training FLOPs ($0.609\times$ versus $0.610\times$), as they are at every setting we tested. The architectures differ in *where* they place their cost, not in total compute efficiency at this scale, and a comparison that fixes only one of the two axes will report whichever answer that axis favours. We report both throughout for this reason.

The control at L=16, with value embeddings matched. The L=8 control sits below MST's amortization threshold, so we repeat it at L=16, again anchoring the budget on MoL's own allocation under our protocol (3.658B tokens, that is 10.5 tokens per total scaling parameter) so that MoL cannot be called undertrained. Both arms now carry their per-block or per-stream value-embedding variant, matching the configuration each paper actually proposes. The budget rule is identical for both arms, but its effect is not: MoL carries $3.74\times$ more total than active matrix parameters against MST's $1.48\times$, so the same rule trains MoL at $2.96\times$ its active-scaled budget and MST at $1.29\times$ its own. The anchor is generous to MoL by construction, which is why we chose it.

Two things must be said with this table. MoL retains the lower absolute BPB, and it leads on training FLOPs ($1.144\times$ against $0.939\times$), consistent with the claim we decline to make below. And the protocol is not symmetric in MST's favour on the per-token axis: anchoring on MoL's budget leaves MST trained $3.13\times$ past its own allocation, and overtraining improves BPB at fixed FLOPs per token. The $1.998\times$ is therefore not comparable to the $1.222\times$ MST reads at L=16 on the ladder; it is comparable only to MoL's $1.849\times$ in the same row, which is the entire purpose of the control. We chose this anchor rather than MST's because the alternative invites the objection that MoL was starved, and their Appendix J pre-registers undertraining as a concern.

What we claim against MoL, and what we do not. We do not claim a total-compute advantage over MoL. On training FLOPs the two are indistinguishable at every setting we measured: $0.665\times$ versus $0.666\times$ at L=8 under each arm's own budget, $1.175\times$ versus $1.181\times$ at L=16, and $0.610\times$ versus $0.609\times$ in the iso-token control, with matching growth across L=8 to L=16 ($1.766\times$ against $1.774\times$). The two architectures place their cost differently; at this scale they do not differ in total compute efficiency.

The difference is in parameters, and it is not marginal. At L=16 MST carries 77.7M matrix parameters against MoL's 314.8M ($0.25\times$), and 413.2M total against 1388.6M ($0.30\times$), each arm with its own value-embedding variant. MoL's 1+3of15 topology instantiates fifteen blocks per layer of which four are active, and pays a $2Dd$ projection wrapper on each; MST's partition pays a single coupling per layer regardless of N (Eq. 3). Together with the iso-token result this gives the claim we do make: *MST matches MoL's total-compute efficiency with four times fewer matrix parameters and a lower per-token cost at equal training data.*

5.4 Separation from MoL

MoL is the closest concurrent work and the comparison deserves to be made on more than one axis. We separate on four, of which two are structural and hold regardless of protocol, and two are measured.

1. The projection overhead is a structural constant, and we measure it. MoL wraps every thin block in its own $W_{\text{down}}/W_{\text{up}}$ pair, so the cost of entering and leaving a narrow block is paid *per block*. MST partitions the residual stream and pays a single coupling per layer. Section 3 gives the closed forms; their consequence is a ratio of total to active matrix parameters that does not improve with scale:

	MoL 1+3of15		MST N=4	
L	matrices	total/active	matrices	total/active
8	39.4M	3.74	10.0M	1.46
16	314.8M	3.74	77.7M	1.48
20	—	—	150.9M	1.48
24	—	—	259.7M	1.49

At L=16 MoL carries $4.05\times$ MST's total matrix parameters to buy $1.60\times$ the active ones. Against the dense curve the two read $0.515\times$ and $1.032\times$ on total matrix parameters, a factor of two. The formula reproduces MoL's own published plumbing fractions (40%, 57%, 73% at d=256, 128, 64) exactly, so this is their arithmetic, not our reinterpretation.

2. At equal data the ranking inverts, and we win it at two thirds the parameters. Every unequal-budget comparison between these architectures is confounded, because compute-optimal budgeting hands MoL $3.75\times$ the data for carrying $3.75\times$ the total parameters. The iso-token control (Table 4) removes it: MST reads $1.061\times$ on FLOPs per token and MoL $0.962\times$, MST above the dense curve and MoL below it, with MST at 60.3M total parameters against MoL's 89.7M. This is the only comparison between the two that fixes the protocol, and it is the one we rest on.

3. Wall clock, on the axis where it is defensible. At a fixed T=2048 neither architecture converts its FLOP saving into time, and we say so in Section 5: MST reaches roughly half dense's hardware utilization, and a step-time comparison at equal depth is uninformative because the dense arm there carries $1.44\times$ the parameters and $3.22\times$ the FLOPs per token. Long-context prefill is different. MST's windowed streams scale as $T^{1.59}$ against dense's $T^{1.85}$, so matched *on quality* MST overtakes dense beyond $T\approx$190K and is $1.31\times$ faster at T=524288, on $4.3\times$ less peak memory (Table 10). MoL reports the opposite for its own sparse path (its Section 5.6): $2.8\times$ slower than its batched fallback on two hardware classes, decode slower than dense at every measured context, and dense still ahead on prefill at $T \leq$ 2K. Both architectures reduce FLOPs; ours turns that into time in the long-context regime, and we do not claim it anywhere else.

4. Their central mechanism does not transfer, which is the sharpest evidence the architectures differ. MoL's Section 3.1 documents an attention coverage failure at high sparsity, and its Section 3.2 introduces an always-active shared block to repair it. MST never gates attention, so it never has the problem. We tested whether adopting their mechanism would help us. At L=8 with all arms on a common token budget, gating attention buys 11.5% of FLOPs per token and costs 0.0244 BPB where the dense exchange rate prices that saving at 0.0131: it **costs** $1.86\times$ **what it saves**, and the multiplier falls from $0.899\times$ to $0.812\times$. Adding their shared stream repairs it to within 7×10^{-5} of the dense curve, confirming their fix works, but only back to neutrality. The result replicates at

Table 6: CORE against cost. The curve $\mathrm{CORE} = a + b \ln x$ is fitted to the three dense points. With three points and two parameters the fit has one residual degree of freedom, so its $R^2 > 0.999$ is close to automatic and is not itself evidence; the evidence is the agreement with the independently derived BPB multipliers in the last column. "Dense at MST's cost" reads the fitted curve at MST L=24's value; the multiplier is the dense cost needed to reach MST's CORE, divided by MST's actual cost, so > 1 favours MST. The final column repeats the BPB multipliers of Table 2 for comparison.

Cost axis	MST L=24	Dense at MST's cost	CORE mult.	BPB mult.
Active matrix params	174.8M	0.1701	$\mathbf{1.36}\times$	—
FLOPs/token	1.481×10^9	0.1753	$\mathbf{1.24}\times$	$1.23\times$
Training FLOPs	4.823×10^{18}	0.1910	$1.07\times$	$1.10\times$
Total matrix params	259.7M	0.2008	$0.92\times$	$0.92\times$
Active params	879.4M	0.2307	$0.69\times$	—
Total params	964.4M	0.2399	$0.63\times$	$0.64\times$

Dense reference points: L=16 (CORE 0.181), L=20 (0.233), L=22 (0.255). MST L=24 scores 0.194. Dense is dense, so its active and total counts coincide and the BPB multipliers were only ever computed on the total axes; the two active rows have no BPB counterpart to compare against and are new here. The ordering is the same one the BPB curves give: MST buys inference compute efficiently, is neutral on total matrix parameters, and pays on total parameters, where the per-stream value embeddings dominate. Active matrix parameters, the axis that both excludes the embeddings MST is charged for and counts only the streams that actually run, is where the architecture looks best at $1.36\times$.

L=16. A mechanism that is load-bearing in one architecture and actively harmful in the other is not the same architecture twice.

What we do not claim. We do not claim a total-compute advantage. On training FLOPs the two are indistinguishable at every setting we measured, and we say so in the paragraph above. We also do not run MoL's headline configuration, which pairs its routed blocks with Gated DeltaNet; our reimplementation is all-softmax, a control they run themselves. Our claim is narrower and, we think, more useful: at equal data MST reaches a better point on the compute–quality frontier than MoL, at a fraction of the parameters, and it does so without the mechanism MoL needs to make its own sparsity viable.

A caution about the per-token axis. Under compute-optimal budgeting an architecture's token allocation scales with its own parameter count, so the FLOPs-per-token axis compares models that saw different amounts of data. This cuts both ways rather than favouring one architecture: at L=16 MST trains on $0.485\times$ dense's tokens and is penalised on that axis, while MoL trains on $3.13\times$ MST's and is flattered by it. We therefore restrict cross-architecture per-token comparisons to the iso-token control of Table 4, and Figure 3 plots MoL on the training-FLOPs and parameter panels only.

5.5 Downstream transfer

Perplexity gaps of this size need not translate into downstream ability, so we evaluate zero-shot transfer against two dense reference points that bracket MST: one matched on total parameters and one matched on FLOPs per token (Table 7).

The head-to-head against the iso-FLOP point is favourable but individually weak: MST wins 12 of the 22 tasks, loses 7 and ties 3, a mean per-task margin of $+0.75$ points with a standard error of 0.54 ($t = 1.41$, sign-test $p = 0.36$), and BoolQ ($+7.9$) and SQuAD ($+6.2$) supply most of it. The result that does carry weight is the agreement in Table 6: placing MST on a CORE-versus-cost curve fitted to three dense points reproduces the BPB-derived Pareto multipliers of Table 2 on all four cost axes, to within 0.03. CORE was not used to fit anything, so this is an out-of-sample check that the BPB advantage is a capability advantage and not a tokenizer or calibration artifact.

Table 7: Zero-shot accuracy (%). MST is compared against a dense model matched on total parameters and a dense model matched on FLOPs per token.

Task	MST L=24 (964M total)	Dense L=20 (iso-param, 0.93×)	Dense L=16 (iso-FLOP, 1.07×)
Active matrices	174.8M	393.2M	201.3M
ARC-Easy (10-shot)	64.1	65.5	61.2
PIQA (10-shot)	70.5	72.9	70.1
HellaSwag (10-shot)	43.5	49.4	42.5
Winograd (0-shot)	61.2	65.2	64.8
LAMBADA (0-shot)	34.9	40.5	33.5
OpenBook QA (0-shot)	34.2	38.6	36.4
ARC-Challenge (10-shot)	33.4	36.9	31.7
CORE metric	**0.194**	**0.233**	**0.181**

Seven of the 22 CORE tasks, chosen as those with meaningful headroom at this scale; the CORE metric aggregates all 22. Tasks on which a model of this size sits at chance (CommonsenseQA, WinoGrande, Jeopardy, BigBench repeat-copy) carry no signal here and are deferred to Appendix D. The reference points bracket MST L=24 (964M total, 259.7M matrices, 1.481×10^9 FLOPs/token): dense L=20 matches it on total parameters to $0.93\times$ and dense L=16 on FLOPs per token to $1.07\times$. We bracket on *total* rather than matrix parameters here because this table fixes a resource a practitioner pays for rather than isolating the cost the architecture is responsible for; the total-matrix-parameter bracket would be L=18 at $1.10\times$. On *active* matrix parameters no dense point in the ladder is low enough to bracket from below: MST L=24 runs 174.8M against dense L=16's 201.3M, so even the iso-FLOP reference carries $1.15\times$ MST's active matrices while scoring lower (0.181 against 0.194). The dense ladder would need $L \approx 15$ for a true iso-active-matrix bracket. MST beats the iso-FLOP point on the aggregate (0.194 against 0.181) and loses to the iso-parameter point (0.233), which is the same ordering the BPB multipliers give and the reason Table 6 reports per-axis multipliers rather than a single verdict.

5.6 Throughput, memory and hardware utilization

FLOP counts are not wall-clock, and at equal depth the two are not comparable either. Table 8 shows MST completing a training step in 420.4 ms against dense's 504.4, but the dense arm there carries $1.44\times$ the parameters and $3.22\times$ the FLOPs per token and reaches 0.070 BPB better, so the step-time gap says only that a smaller model runs less efficiently. The comparable quantity is utilization, and on that MST reaches 30.0% against dense's 61.6%: it issues $2.46\times$ fewer FLOPs and uses the machine half as well. We do not claim faster training. The wall-clock axis on which we do claim an advantage is long-context prefill, below, where the arms are matched on quality rather than on depth.

The utilization gap decomposes, and only part of it is ours to fix. We measured the block-diagonal penalty directly rather than inferring it from FLOP counts, racing five alternatives against `cuBLAS` on the exact FFN shapes at L=24, N=4 and M=16384 on an H200. Two candidate explanations die immediately: it is not bandwidth, since MST sits at 302 FLOP/byte against the device's roofline ridge of 206 and both arms are compute-bound, and it is not wave quantization, since MST emits the same 6144 output tiles as dense at 99% wave efficiency.

The mechanism is the depth of the K loop. Partitioning drops K from D to D/N, so each output tile runs three K-steps where dense runs twelve and the same prologue and epilogue amortize over a quarter of the work. The measurement tracks that monotonically: the FFN up-projection, with 3 K-steps against dense's 12, reaches $0.707\times$ dense's throughput, and the down-projection, with 12 against 48, reaches $0.772\times$.

The penalty is irreducible with available kernels. Against `torch.bmm` on the current layout we measured Inductor's `max-autotune` at 0.776/0.793×, an explicit sweep of Triton configurations over block shape, pipeline stages and warps at 0.701/0.809×, `grouped_mm` at 0.815/0.693×, a per-stream loop at 0.855/0.918×, and a pre-transposed weight at 0.949/1.008×. Inductor's autotuner

[TODO: Fig. 3: (a) fused attention throughput vs d_h at constant total width: 171.9 / 283.9 / 365.7 TFLOP/s for $d_h = 32/64/128$; (b) MST kernel-time breakdown.]

Figure 4: The utilization gap decomposes into a kernel effect and an implementation cost. Fused attention at d_h=32, which N=4 forces, reaches only 2.13× less throughput than at d_h=128 with the total attention width held fixed and no MST code involved.

selects `cuBLAS` over all 19 of its own Triton candidates. A hand-written kernel would have to beat NVIDIA's tuned kernels on their own shapes, so we do not claim one is forthcoming.

What this leaves is a decomposition. The GEMM penalty is 0.739 while model-level utilization is 0.462, so 62% of the gap is the shape of the matrices and 38% is our own non-GEMM overhead: the layout permutations, per-stream normalizations and rotary application, the router, and the transition. The two halves differ in kind. The first is a property of running N narrow GEMMs and does not improve with better software; it also worsens with larger GPUs, measuring 0.948 on a 20-SM device against 0.739 on a 132-SM one, since it is an occupancy effect. The second is implementation, and hoisting the shared rotary and QK-norm work out of the per-stream loop already recovered 3.5%. Closing it entirely would move utilization from 0.462 to 0.739 and the equal-compute wall clock from 2.71× to 1.69×. We report the split rather than the aggregate because the two admit different remedies, and we do not claim to have removed the second.

Memory. MST's peak memory is 8% *higher* than dense despite 3.2× fewer per-layer parameters. The cause is the batched formulation rather than the partition: applying N stacked weight matrices requires a permutation before the batched matrix multiply, which forces a contiguous copy of a full activation tensor, and this happens for each of the six batched projections per layer. Parameter count shrinks with N; activation traffic does not. The same extra data movement lowers arithmetic intensity, so the memory overhead and the utilization gap share a cause rather than one explaining the other.

Long-context prefill is where the FLOP advantage becomes time. The step-time comparisons above are at a fixed T=2048, where MST's per-stream dispatch cost is amortized over too little work. Prefill at long context is the regime the partition was built for, because multi-scale windows make MST's attention grow far more slowly in T than dense's. Measured on an H200, the local exponent $\mathrm{d}\log(\text{latency})/\mathrm{d}\log T$ between T=131072 and T=262144 is 1.41 for MST at both L=16 and L=24, against 1.69–1.71 for dense at L=16, 18, 20, 22. Dense keeps four full-context layers of width D; MST keeps one full-context *stream* of width D/N in one layer of four, so its quadratic term carries roughly a sixteenth of the weight.

The consequence is a crossover rather than a constant gap, and the gap keeps widening past it (Table 10). Matched on quality, MST at L=24 runs at 0.70× dense's prefill speed at T=65536, 0.88× at T=131072, 1.10× at T=262144 and **1.31**× at T=524288; the curves cross near $T\approx$190K, bracketed by measurements on either side. Matched on FLOPs per token the crossover is later and also reached, at 1.11× by T=524288; matched on parameters MST is ahead at every length, reaching 2.06×. This is the one wall-clock axis on which we claim an advantage, and it is not an artifact of equal-depth comparison: the dense arm here is the one that reaches MST's BPB.

The same asymmetry appears in memory, which we did not anticipate. At T=524288 MST L=24 peaks at 20.7 GB against the quality-matched dense arm's 88.5, a factor of 4.3. Attention over a 32- or 127-token window needs a working set that does not grow with T, and three of MST's four streams are in that regime at every layer, where dense holds full context in one layer of four at the full D. The practical consequence is that the context length at which prefill stops fitting on one device is several times further out for MST.

Figure 4 attributes the utilization gap to the head-dimension regime rather than to the architecture, and therefore identifies it as implementation headroom.

Table 8: Measured throughput and memory. Weights are randomly initialized; these are architecture-level measurements and do not depend on training.

Measurement (L=24, H200)	MST	Dense	
Training step, fwd+bwd (ms)	**420.4**	504.4	MST 1.20× faster
Achieved throughput (TFLOP/s)	297	609	
Model FLOPs utilization (%)	30.0	61.6	dense 2.05× higher
Peak memory (GB)	115.3	106.7	MST 8% higher
KV cache per layer	D	D	identical by construction
Decode latency (ms/token)	--	--	**[TODO: pending]**

Batch 32, T=2048, `torch.compile` enabled for both arms as in training, minimum over 20 iterations. Weights are randomly initialised: these are architecture-level measurements independent of the trained values.

Table 9: Long-context prefill latency (ms, batch 1, H200, `torch.compile` on). Dense L=17 is the arm that reaches MST L=24's BPB (0.7869 against dense's 0.7988 at L=16 and 0.7755 at L=18) and is interpolated geometrically between the two measured neighbours. Ratios above 1 mean MST is faster. The output head is applied to all T positions here, as `forward` computes it; autoregressive prefill projects only the final position and discards the rest, which is 13–16% of forward compute at these lengths and is near-identical work for both arms.

	T=65536	T=131072	T=262144	exponent
MST L=16	82.74	196.28	521.70	1.41
MST L=24	186.40	447.14	1192.39	1.42
dense L=16	114.45	339.68	1108.78	1.71
dense L=18	158.36	475.17	1541.10	1.70
dense L=20	191.91	553.01	1779.04	1.69
dense L=22	248.43	722.62	2326.53	1.69
MST L=24 against its matched dense arm (higher is better for MST)				
matched quality (L=17)	0.72×	0.90×	**1.10×**	
matched FLOPs/tok (L=16)	0.61×	0.76×	0.93×	
matched params (L=21)	1.17×	1.41×	1.71×	

6 Ablations

Ablations are run at L=8 (D=512), where a full grid is affordable, with three seeds per condition. A L=16 grid would cost roughly 40× as much and we did not run one.

Ablation transfer. Small-scale ablations are only useful if their conclusions survive at the scales the paper actually claims. We can test this directly for the one intervention we measured at two depths: the full training recipe of Section 3.3 applied to the plain partition. It preserves its sign and, to within 15%, its magnitude (Table 11). We take this as evidence that the L=8 ordering is informative about L=16, while noting that one intervention at two depths is a weak test and that we cannot rule out an ablation whose sign flips with scale.

Per-stream value embeddings, and where they are paid for. The one component measured at L=16 under the shipped sparse configuration is the width of the value-embedding table. Giving each stream its own d-wide slice of an Nd-wide table, rather than broadcasting one d-wide vector to all streams with per-stream gates, is worth 0.0059 BPB (0.8702 against 0.8761). Since value embeddings are a lookup, this is free on both FLOPs axes: it moves the FLOPs-per-token multiplier from 1.10× to 1.18× and the training-FLOPs multiplier from 0.88× to 1.01× at no compute cost.

It is not free in parameters. The wide table adds 201.3M, which is 1.95× MST's total (Table 12). Against the dense model of matching quality (dense L=12, 0.8654 BPB, 286.3M total) the wide-table configuration is 1.44× larger and the shared-table configuration is 0.74×, so this one choice decides the sign of the total-parameter comparison. It does not touch the transformer-matrix compar-

Table 10: Long-context prefill with the *prefill-accurate* head: `lm_head` is applied to the final position only, matching what autoregressive prefill does (`engine.py` discards the other rows). Latency in ms and peak allocation in GB, batch 1, H200, `torch.compile` on. Dense L=17 is the quality-matched arm, interpolated geometrically between the measured L=16 and L=18. Removing the discarded head is ratio-neutral: at T=262144 the matched-quality ratio is 1.096× either way (cf. Table 9), so it costs both arms the same fraction and only changes what is measurable.

	T=65K	T=131K	T=262K	T=524K	exponent
latency (ms)					
MST L=16	74.44	179.79	484.77	1499.34	1.63
MST L=20	121.21	290.17	785.60	2388.46	1.60
MST L=24	176.27	427.46	1152.22	3467.01	1.59
dense L=16	104.38	318.89	1065.36	3834.22	1.85
dense L=18	146.69	445.98	1497.03	5392.30	1.85
dense L=20	180.85	531.93	1731.52	6120.20	1.82
dense L=22	234.61	698.06	2276.74	8641.42	1.92
MST L=24 against its matched dense arm (higher is better for MST)					
matched quality (L=17)	0.70×	0.88×	1.10×	**1.31×**	
matched FLOPs/tok (L=16)	0.59×	0.75×	0.93×	**1.11×**	
matched params (L=21)	1.15×	1.41×	1.70×	**2.06×**	
peak allocation (GB)					
MST L=24	4.7	6.9	11.5	**20.7**	
dense L=17	13.9	24.5	45.8	88.5	

Table 11: Ablation transfer. The one intervention measured at two depths keeps its sign and approximate magnitude, which is the basis on which we run the remaining ablations at L=8 only. **[TODO: These four values are from the previous aggregate-distribute lineage and no longer match Table 1: the L=16 recipe row reads 0.8810 here against 0.8702 in the cost table. Re-run both conditions in the current lineage, or state explicitly that this table measures the earlier configuration.]**

Condition	L=8 (BPB)	L=16 (BPB)	
Plain partition (no recipe)	1.0798	0.9057	
+ full recipe (Section 3.3)	1.0510	0.8810	
Δ	−0.0288	−0.0247	**same sign,** 0.86× **magnitude**

ison at all — 77.7M against dense's 84.9M, 0.91×, either way — which is why we report matrices separately throughout and treat them as the parameter axis the layer decomposition is actually responsible for. We keep the wide table in the headline configuration because it is free on the compute axes, and report the shared-table point as the operating point for a parameter-constrained deployment.

Granularity. N=4 substantially outperforms N=8 at equal D (−− versus −− BPB). This matches MoL's independent finding that block *width* dominates block *count* at fixed parameters, and runs opposite to the fine-grained-expert trend in FFN-MoE (Dai et al., 2024). Both observations are consistent with the interpretation that when a block contains attention as well as an FFN, per-block capacity matters more than combinatorial flexibility.

Enriching the coupling does not help. Beyond the routed bottleneck we tried a nonlinear (SiLU) bottleneck, an input-gated router, a concat–MLP–split transition, a bilinear second-order aggregate, per-slice independent routing, DenseFormer-style multi-layer lookback, EMA hyper-connected sub-residuals, cross-sub FFN gating, per-token cross-sub KV injection, low-rank cross-sub query modulation, cyclic feature permutation across subs, and a D-wide global residual stream. None improved on the routed bottleneck (Appendix B reports all twelve). Removing the coupling entirely, however, costs 0.039 BPB. Coupling is necessary; its form appears not to be. This mirrors Ternovtsii & Bilak

Table 12: Value-embedding width at L=16, single seed, both arms otherwise the shipped sparse configuration. Multipliers are against the dense ladder; "vs dense" compares total parameters to the dense model of matching quality (L=12, 286.3M).

Value-embedding table	BPB	FLOPs/tok	train FLOPs	Total	Matrices
Per-stream, Nd wide (ours)	**0.8702**	**1.18×**	**1.01×**	413.2M (1.44×)	77.7M
Shared, d wide	0.8761	1.10×	0.88×	**211.9M (0.74×)**	77.7M
Δ	−0.0059	+0.08	+0.13	+201.3M (1.95×)	unchanged

Table 13: Ablations at L=8, D=512, N=4, mean ± s.d. over 3 seeds. Negative Δ is better.

	Condition	BPB	Δ
Controls	Dense baseline (d_h=128)	0.9592 ± 0.0002	n/a
	Dense baseline, d_h=32 *(head-dim control)*	0.9760 ± 0.0003	+0.0168
Coupling	No coupling (independent columns)	--	--
	Parameter-free mean coupling	--	--
	Routed bottleneck (ours)	--	n/a
Recipe	Plain partition (no recipe)	--	--
	+ gradient equalization	--	--
	+ block-diagonal Muon & per-sub LR	--	--
	+ wide transition	--	--
Full	All of the above	--	--
	+ multi-scale sub-windows (**MST**)	--	--
Granularity	N=2	--	--
	N=4	--	n/a
	N=8	--	--

[TODO: Tier-3 grid; the head-dim control is the load-bearing row] Block-diagonal Muon and the per-sub learning rate are ablated as a unit because blocking divides the effective rate by $\sqrt{N}$ and the multiplier restores it; the two separately, and the bottleneck-width sweep, are in Appendix C.

(2026a), who reach an equifinality conclusion for routing topology in sparse MoE, and extends it to a dense partitioned setting.

7 Analysis

The 7.2× gradient imbalance of Section 3.3 raises the concern that a partitioned model silently degenerates into a narrower one. Two inference-time probes on the L=32 model say otherwise (Figure 5).

Every stream is load-bearing. Zeroing any single stream at every layer raises bits-per-byte from 0.710 to between 5.15 and 8.48: the model is destroyed whichever stream is removed. The intervention is deliberately blunt, since a quarter of the residual stream is deleted at every layer, so the magnitude is uninformative and the *spread* is the result. Damage varies by only 1.75× across the four streams, against the 7.2× gradient imbalance the recipe was built to correct. No stream is close to redundant.

Streams divide along token frequency. Attributing the damage of the previous probe token by token, and normalizing each stream by its own mean so that overall importance is factored out, gives a monotonic split across frequency quartiles (Figure 5b). Relative to its own average, stream 2 matters 1.78× more than usual on the rarest quartile and 0.30× on the commonest, a 5.9× swing; stream 1 runs the other way, 0.58× to 1.40×. All four profiles are monotonic. An independent probe agrees: decoding each stream alone through the output head, stream 1 attains 3.52% top-1 agreement with the full model against 0.03–0.24% for the others (Figure 5c), and the tokens it promotes are

dominated by punctuation and line breaks, which is the highest-frequency class. Two probes with no mechanism in common therefore place the same stream at the same end of the same axis.

We report this as evidence that the partition does something structured rather than as an interpretability claim, and one caveat limits how far it can be pushed: deleting a stream raises token loss by roughly 20 nats, well above the $\log V = 10.4$ nats of a uniform predictor, so the perturbed model is confidently wrong rather than merely uninformed. The comparison is between catastrophic failures, and the monotonicity is what carries the signal. A graded perturbation would support a stronger claim.

In weight space the streams are genuinely distinct. The activation statistic above compares vectors living in different channel ranges, which limits what it licenses. The N distribute matrices do not have that problem: they all map from the *same* aggregate vector, so cosine between them is well posed. Mean pairwise cosine among the W_i^D is $+0.040$ overall and decays from $+0.079$ at the first layer to $+0.015$ at the last, while the per-stream W^Q, W^K, W^V and W^{F1} sit at $|\cos| < 0.0015$, indistinguishable from the ≈ 0.002 expected of unrelated matrices of this size (Figure 5f). The symmetry breaking of Section 3 works, and the streams diverge further with depth.

Streams stay near-orthogonal, and the coupling summarizes rather than selects. Mean pairwise cosine similarity between streams is close to zero through the body of the network, between -0.09 and $+0.08$ for layers 8–19, rising to $+0.64$ at layer 2 and to $+0.16$ near the output. It does not approach the $-1/(N-1) = -0.333$ floor that maximal mutual repulsion would produce: the streams are approximately independent rather than actively anti-correlated, and they grow more alike at both ends of the network, where they sit closest to the shared embedding and the shared output head. Router entropy stays between 1.21 and 1.38 against a maximum of $\log N = 1.386$, within 13% of uniform at every layer, so the coupling acts as a summarizer rather than a selector.

Two caveats on these diagnostics. First, near-uniform routing is partly *enforced*: the load-balancing term of Section 3 penalizes $\sum_i \bar{p}_i^2$ and is minimized at the uniform p, so high entropy is evidence that the regularizer works, not that a free router would choose to average. What the entropy does establish is that whatever the router contributes (the parameter-free mean ablation of Section 6 puts that at 0.039 BPB) it contributes through small deviations from uniform rather than through selection. Second, the similarity statistic compares stream i's mean direction with stream j's, but the two vectors occupy different channel ranges of the model and there is no reason for index k to denote the same feature in both. The measure is therefore asymmetric in what it licenses: a high value is strong evidence of redundancy, whereas a value near zero is only consistent with independence and does not establish it, since two streams could show zero mean-direction similarity while being strongly correlated token by token. The informative datum here is the $+0.64$ at layer 2, not the near-zero plateau.

Device parallelism. Streams have no within-layer cross-stream data dependency, so the N columns of a layer can be placed on different devices with a single d-wide all-reduce at the coupling. That exchange is narrower than the two D-wide all-reduces per layer of tensor parallelism (Narayanan et al., 2021), and it avoids the all-to-all token dispatch that expert parallelism requires. **[TODO: optional: measured 2-GPU numbers]**

8 Limitations

Range of validity. The efficiency multipliers of Table 2 are measured from 413M to 964M total parameters and 6.7×10^{17} to 4.8×10^{18} training FLOPs, and rest on *three* MST points against eleven dense points. That is the principal limitation of this paper. The L=20 point is off-trend on every axis ($1.15\times$ FLOPs per token against $1.25\times$ and $1.30\times$ at L=16 and L=24), and with three points we cannot distinguish a genuine irregularity from run-to-run variation; the MST fits in Table 3 carry $R^2 \approx 0.98$ against dense's 0.999. At L=8 MST is *worse* than dense on both compute axes ($0.87\times$ FLOPs per token): the coupling and input/output projections are a fixed overhead that has not yet amortized. We cannot exclude a crossover beyond 964M. This caution is not hypothetical: MoL reports an advantage at 104M that inverts into a 3.01 PPL deficit at 1.3B on the iso-total-parameter axis.

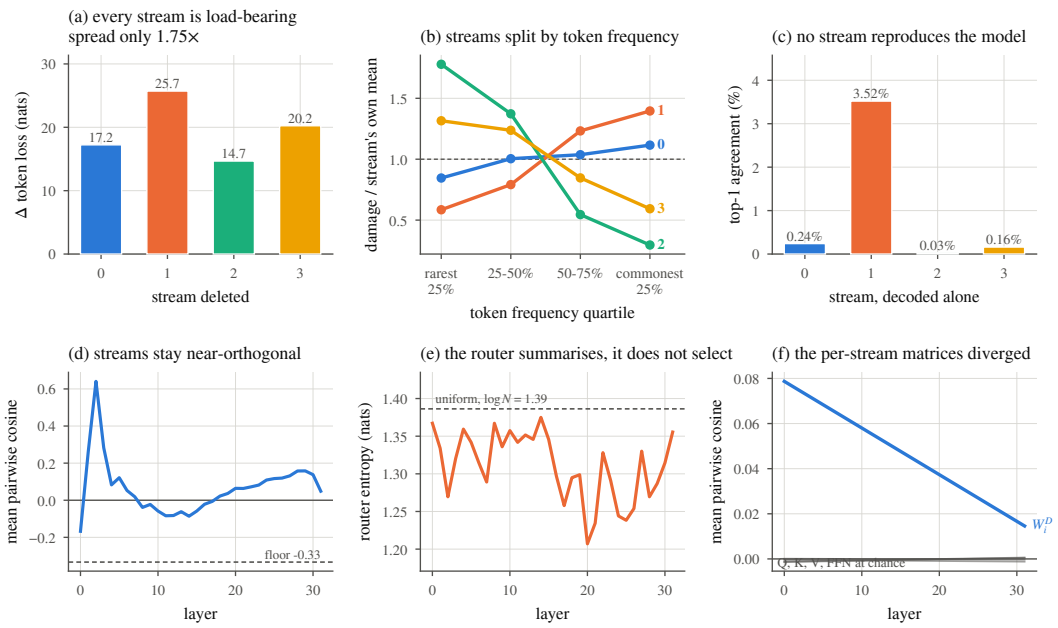

Figure 5: Stream-level analysis of the L=32 model. **(a)** deleting any stream destroys the model, and the spread across streams is only 1.75×. **(b)** normalising each stream by its own mean reveals a monotonic split along token frequency: streams 2 and 3 carry rare tokens, streams 0 and 1 carry common ones. **(c)** decoded alone, no stream reproduces more than 3.5% of the model's predictions. **(d,e)** activation similarity and router entropy per layer, against the $-1/(N-1)$ floor and $\log N$. **(f)** in weight space the per-stream attention and FFN matrices are uncorrelated at every depth, while the distribute matrices share a small component that decays with depth.

Token ratio. All models are trained at 10.5 tokens per scaling parameter. Partial evidence that the result is not an artifact of this ratio comes from overtrained runs at 2.6× the budget, where the ordering is preserved **[TODO: report these two points; a dense point at matched ratio would make the claim clean]**.

Hardware utilization and decode. MST attains half the utilization of dense, for the two reasons decomposed in Section 5: a measured 2.13× kernel penalty intrinsic to d_h=32, and an implementation cost we have not removed. Our wall-clock advantage is therefore smaller than our FLOP advantage. Decode is worse still: it is the regime where per-stream dispatch is least amortized, since there is no sequence dimension over which to hide the N-fold increase in kernel launches, and we measure MST slower than dense there (Table 8). We claim fewer FLOPs per token and faster training; we do not claim faster inference, and a deployment dominated by autoregressive decoding would not benefit from MST as implemented here.

Single seed at scale. Seed variance is characterized at L=8 only, where it is $\sigma \leq 0.0003$ BPB over three seeds, two orders of magnitude below every effect we report; the $L \geq 16$ ladder is single-seed per point, as is standard at this scale but worth stating.

Scope. We study pre-training perplexity and zero-shot transfer for decoder-only language models at T=2048. We do not study instruction tuning, long-context training, or N scaled jointly with D.

9 Conclusion

Narrow parallel blocks are a promising way to restructure the transformer layer, but the way they are usually built, projecting into and out of a full-width residual stream and paying for the projections with sparsity, gives up total-parameter efficiency to buy active-compute efficiency. We showed that if the narrow blocks partition the residual stream instead, the decomposition is free: attention cost and KV cache are preserved exactly, per-layer matmul parameters fall by N, no dispatch machinery

is needed, and the resulting dense model dominates a strongly-tuned dense baseline on parameters, inference FLOPs and training compute at once. The remaining design question is not how to route, but how much the columns must talk. Our evidence is that they must talk, and that almost any way of doing so works equally well.

AI USE STATEMENT

Generative AI tools were used in this work as a coding and drafting assistant. Specifically: (i) implementation support for the architecture, training and evaluation code; (ii) analysis scripts that recompute the parameter and FLOP accounting reported in Table 1 and fit the scaling laws in Table 3; (iii) figure generation; and (iv) drafting and editing of the manuscript text. **[TODO: authors: confirm or amend this list, and add any use not covered, e.g. literature search or idea generation]**

Generative AI was not used to generate or alter any experimental result. All numbers reported in this paper come from training runs and from deterministic accounting code executed by the authors; no measurement was produced, imputed or edited by a language model. All AI-assisted code was read, executed and checked against independent references by the authors: the parameter and FLOP accounting was verified against the closed-form derivation of Appendix A, and the scaling fits were reproduced from the raw run table. The authors have reviewed all AI-assisted text and take full responsibility for the final content of this work, including all text, claims and artifacts.

ETHICS STATEMENT

This work studies the pre-training efficiency of decoder-only language models. It involves no human subjects, no personally identifying data and no user study. Training data is a publicly released filtered web corpus (Penedo et al., 2024); we introduce no new dataset and release none. The models trained here are small by contemporary standards, are not instruction-tuned or deployed, and are not intended or suitable for downstream use; they inherit whatever biases are present in the underlying corpus, and we make no claim that they are safe for application. The contribution is architectural, and its foreseeable effect is to reduce the parameter count and compute required to reach a given language modelling quality. We regard the primary societal consideration as the standard dual-use property of any efficiency result: the same reduction that lowers the cost of research also lowers the cost of producing capable models. We see no concern specific to this architecture beyond that general property. The authors declare no conflicts of interest.

REPRODUCIBILITY STATEMENT

The architecture is specified completely in Section 3: the per-stream sublayers, the coupling of Eq. 2, the geometric window assignment of Section 3.2, and the two optimizer modifications of Section 3.3, with the term-by-term parameter budget in Appendix A. Section 4 fixes the model ladder ($D = 64L$ rounded to a multiple of 128, $N{=}4$, $T{=}2048$), the token budget rule (10.5 tokens per scaling parameter, where scaling parameters are transformer matrices plus the output head), and the fact that the dense baseline shares every component and hyper-parameter with MST except the layer structure. Full hyper-parameters and per-run configurations are in Appendix D.

Two points bear directly on reproducing the numbers. First, all cost columns in Table 1 are recomputed for both arms from a single implementation at one commit rather than taken from the original run logs, so the two arms are comparable to each other; the accounting script and the resulting table are released with the code. Second, the ablations of Section 6 are run at $L{=}8$ with three seeds and we report the seed spread, while the $L \geq 16$ ladder is single-seed per point; Table 11 gives the evidence we have that the $L{=}8$ ordering transfers. **[TODO: authors: add the code/artifact URL once anonymised, and state the exact commit]**

REFERENCES

Noah Amsel, David Persson, Christopher Musco, and Robert M Gower. The polar express: Optimal matrix sign methods and their application to the muon algorithm. *arXiv preprint arXiv:2505.16932*, 2025.

Anonymous. The dual-stream transformer: Channelized architecture for interpretable language modeling. *arXiv preprint arXiv:2603.07461*, 2026a.

Anonymous. Revisiting the shape convention of transformer language models. *arXiv preprint arXiv:2602.06471*, 2026b.

Sangmin Bae et al. Mixture-of-recursions: Learning dynamic recursive depths for adaptive token-level computation. In *NeurIPS*, 2025.

Cenk Baykal, Dylan Cutler, Nishanth Dikkala, Nikhil Ghosh, Rina Panigrahy, and Xin Wang. Alternating updates for efficient transformers. In *NeurIPS*, 2023.

Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

Shane Bergsma et al. Power lines: Scaling laws for weight decay and batch size in llm pre-training. *arXiv preprint arXiv:2505.13738*, 2025.

Mikhail S Burtsev and Anna Rumshisky. Multi-stream transformers. *arXiv preprint arXiv:2107.10342*, 2021.

Róbert Csordás, Kazuki Irie, Jürgen Schmidhuber, Christopher Potts, and Christopher D Manning. Moeut: Mixture-of-experts universal transformers. In *NeurIPS*, 2024.

Damai Dai et al. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066*, 2024.

Tri Dao, Beidi Chen, Nimit S Sohoni, Arjun Desai, Michael Poli, Jessica Grogan, Alexander Liu, Aniruddh Rao, Atri Rudra, and Christopher Ré. Monarch: Expressive structured matrices for efficient and accurate training. In *ICML*, 2022.

William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *JMLR*, 23:1–39, 2022.

Tianyu Fu et al. Moa: Mixture of sparse attention for automatic large language model compression. *arXiv preprint arXiv:2406.14909*, 2024.

Jordan Hoffmann et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks. `https://kellerjordan.github.io/posts/muon/`, 2024.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

Andrej Karpathy. nanochat: The best chatgpt that $100 can buy. `https://github.com/karpathy/nanochat`, 2025.

Changwoo Lee et al. Blast: Block-level adaptive structured matrices for efficient deep neural network inference. In *NeurIPS*, 2024.

Deepak Narayanan et al. Efficient large-scale language model training on gpu clusters using megatron-lm. In *SC*, 2021.

Guilherme Penedo et al. The fineweb datasets: Decanting the web for the finest text data at scale. *NeurIPS Datasets and Benchmarks*, 2024.

David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. In *ICML*, 2024.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *ICLR*, 2017.

Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

Sainbayar Sukhbaatar, Edouard Grave, Piotr Bojanowski, and Armand Joulin. Adaptive attention span in transformers. *ACL*, 2019.

Ivan Ternovtsii and Yurii Bilak. Equifinality in mixture of experts: Routing topology does not determine language modeling quality. *arXiv preprint arXiv:2604.14419*, 2026a.

Ivan Ternovtsii and Yurii Bilak. Mixture of layers with hybrid attention: Parallel thin blocks for sparse transformer compute. *arXiv preprint arXiv:2605.09516*, 2026b.

Xun Wu, Shaohan Huang, Wenhui Wang, and Furu Wei. Multi-head mixture-of-experts. In *NeurIPS*, 2024.

Xie et al. Manifold-constrained hyper-connections. *arXiv preprint*, 2026.

Greg Yang, Edward J Hu, Igor Babuschkin, Szymon Sidor, Xiaodong Liu, David Farhi, Nick Ryder, Jakub Pachocki, Weizhu Chen, and Jianfeng Gao. Tensor programs v: Tuning large neural networks via zero-shot hyperparameter transfer. *arXiv preprint arXiv:2203.03466*, 2021.

Yujiao Yang, Jing Lian, and Linhui Li. Union of experts: Adapting hierarchical routing to equivalently decomposed transformer. *arXiv preprint arXiv:2503.02495*, 2025.

Minshen Zhang, Xiang Hu, Jianguo Li, Wei Wu, and Kewei Tu. Flash multi-head feed-forward network. *arXiv preprint arXiv:2512.06989*, 2026.

Yixing Zhang et al. Mswa: Refining local attention with multi-scale window attention. *arXiv preprint arXiv:2501.01039*, 2025.

Zhanchao Zhou et al. Value residual learning for alleviating attention concentration in transformers. *arXiv preprint arXiv:2410.17897*, 2024.

Defa Zhu, Hongzhi Huang, Zihao Huang, Yutao Zeng, Yunyao Mao, Banggu Wu, Qiyang Min, and Xun Zhou. Hyper-connections. In *ICLR*, 2025.

A PER-LAYER PARAMETER BUDGET

Term by term, for one layer at width D with N streams of width $d = D/N$. This is the derivation behind Eq. 3.

Table 14: Per-layer parameter budget. The N sub-transformers cost $12D^2/N$ and the coupling costs a further $3D^2/N$, so a layer costs $15D^2/N$ against $12D^2$ dense: a reduction of $0.8N$, not N. The router term Nd is 0.02% of the layer at D=512 and is omitted from the totals.

Component	Shapes	Parameters	At N=4
Attention, N subs	$4 \times (d \times d)$ each	$4D^2/N$	$1.00\,D^2$
FFN, N subs	$(4d \times d) + (d \times 4d)$ each	$8D^2/N$	$2.00\,D^2$
Coupling bottleneck	$(D \times d) + (d \times D)$	$2D^2/N$	$0.50\,D^2$
Coupling scatter	$N \times (d \times d)$	D^2/N	$0.25\,D^2$
Coupling router	$N \times d$	Nd	≈ 0
MST layer		$\mathbf{15D^2/N}$	$\mathbf{3.75\,D^2}$
Dense layer	$4 \times (D \times D) + 8D^2$	$12D^2$	$12.00\,D^2$
Reduction		$\mathbf{0.8N}$	$\mathbf{3.20\times}$

B COUPLING VARIANTS THAT DID NOT HELP

Every variant below was run on top of the full recipe of Section 3.3 at L=8, three seeds each. None improved on the routed bottleneck of Eq. 2. We report them because the pattern is itself the finding: the coupling must exist, but its form is close to irrelevant.

Table 15: Coupling variants that did not improve on the routed bottleneck, L=8.

Hypothesis	Variant	Δ BPB	Variant	Δ BPB
Needs nonlinearity	SiLU bottleneck	−−	Concat–MLP–split	−−
Needs finer routing	Per-slice routing	−−	Input-gated router	−−
Needs memory	Multi-layer lookback	−−	Hyper-connected residual	−−
Needs 2nd order	Bilinear aggregate	−−	Cross-sub FFN gating	−−
Attention needs cross-sub	Cross-sub KV injection	−−	Low-rank query modulation	−−
Features stuck in one sub	Cyclic feature permutation	−−	Global residual stream	−−

C ADDITIONAL ABLATIONS

Table 16: Rows omitted from Table 13: the two optimizer interventions applied separately, and the transition bottleneck width sweep. L=8, three seeds.

	Condition	BPB	Δ
Optimizer, separately	Block-diagonal Muon only	−−	−−
	Per-sub LR only	−−	−−
Bottleneck width	×1 (d)	−−	−−
	×2	−−	−−
	×4 (D)	−−	n/a

D EXPERIMENTAL DETAILS

[TODO: full hyper-parameters; per-run configuration hashes; extended ablation table; FLOP accounting derivation; seed-variance table]